

Examen Parcial: Docker A

CU

Dura **25 minutos** y vale **10 puntos**. Sin apuntes, formularios ni dispositivos electrónicos: sólo estas hojas, pluma o lápiz y goma. Prohibido copiar o hablar, también al terminar. La clase se reanuda a las **19:30**. Dos ejercicios, uno por cara.

1. Del repo a la imagen (4 puntos)

<pre>clima/ ← estás aquí ├── Dockerfile ├── README.md ├── requirements.txt ├── app/ │ ├── main.py │ └── utils.py └── data/ └── clima.csv</pre>	clima/ es un repo de Python. El programa es app/main.py (usa app/utils.py). En tu máquina los datos están en data/; dentro del contenedor el programa los lee de /clima/data y escribe ahí un resumen. Queremos la imagen clima:1, así: • dentro de la imagen se trabaja en /clima; ahí van /clima/requirements.txt y /clima/app/; • al arrancar, el contenedor corre python app/main.py parado en /clima; • data/ no entra a la imagen: al correr, se monta en /clima/data.
---	---

Completa el Dockerfile. En cada COPY, el origen es una ruta **relativa** a clima/ (el contexto del build) y el destino es una ruta **absoluta** dentro de la imagen. **(1 punto, 0.25 c/u)**

FROM	python:3.12-slim
WORKDIR	(a)_____
COPY	requirements.txt (b)_____
RUN	pip install -r requirements.txt
COPY	(c)_____ /clima/app
CMD	(d)_____

Escribe cada comando completo. Tu terminal está en clima/.

e) Construye la imagen clima:1. (0.5)

\$

f) El programa lee los datos de /clima/data. Corre clima:1 en un contenedor que se borre al terminar y que monte tu carpeta data/ en /clima/data. (0.5)

\$

g) Abre una terminal bash en un contenedor nuevo de clima:1, que se borre al salir, para revisar /clima (escribe sólo el comando que abre la terminal). (0.5)

\$

h) Con el Dockerfile ya completo, ¿qué archivos del repo quedan dentro de la imagen? Márcalos con X. (0.5)

☐ Dockerfile	☐ README.md	☐ requirements.txt
☐ app/main.py	☐ app/utils.py	☐ data/clima.csv

i) Alguien entra a la carpeta clima/app/ (cd app) y desde ahí corre docker build -t clima:1 -f ../Dockerfile . ¿El build termina bien? ¿Por qué? (0.5)

j) En f), el programa escribe /clima/data/resumen.csv y luego el contenedor se borra. ¿Dónde existe ahora ese archivo: en tu carpeta data/, en la imagen o en ningún lado? ¿Y si lo hubiera escrito en /clima/resumen.csv? (mismas opciones) (0.5)

Examen A · 2. ¿Qué cambió? (6 puntos, 0.5 c/u)

En lab/ hay sólo este Dockerfile y nota.txt, cuyo texto es la palabra **uno**. No existen imágenes, contenedores, volúmenes ni caché previos. Los comandos se corren **en orden**, seguidos y sin pausas (ningún sleep 600 alcanza a terminar), todos desde lab/: cada fila parte de lo que dejó la anterior. Contesta con **una palabra o frase corta**; si crees que un comando falla, escribe **error**.

```
FROM alpine:3.20
WORKDIR /w
COPY nota.txt .
CMD ["cat", "nota.txt"]
```

#	Comandos (en orden)	Pregunta	Respuesta
1	<pre>docker build -t nota:1 . docker run -d --name c1 nota:1 sleep 600 docker exec c1 sh -c 'echo dos > nota.txt'</pre>	¿Qué contiene nota.txt en tu carpeta lab/?	
2	<pre>docker build -t nota:1 .</pre>	El paso COPY, ¿se vuelve a ejecutar o sale de la caché? ¿Por qué? (pocas palabras)	
3	<pre>echo tres > nota.txt docker run --rm nota:1 docker exec c1 cat nota.txt</pre>	¿Qué imprime el run?	
		¿Y el exec?	
4	<pre>docker build -t nota:1 . docker run --rm nota:1</pre>	¿Qué imprime el run?	
5	<pre>docker run -d --name c2 -v "\$(pwd)":/w nota:1 sleep 600 docker exec c2 sh -c 'echo cuatro > nota.txt'</pre>	¿Qué contiene nota.txt en tu carpeta lab/?	
6	<pre>docker run --rm nota:1</pre>	¿Qué imprime?	
7	<pre>echo cinco > nota.txt docker exec c2 cat nota.txt</pre>	¿Qué imprime el exec?	
8	<pre>docker rm -f c1 c2</pre>	El texto dos , ¿dónde existe ahora? (tu carpeta / la imagen / en ningún lado)	
9	<pre>echo seis > extra.txt docker run --rm -v "\$(pwd)/extra.txt":/w/nota.txt nota:1</pre>	¿Qué imprime?	
10	<pre>docker build -t nota:1 . docker run --rm -v almacen:/w nota:1</pre>	¿Qué imprime el run?	
11	<pre>echo siete > nota.txt docker build -t nota:1 . docker run --rm -v almacen:/w nota:1</pre>	¿Qué imprime el run?	

Examen Parcial: Docker B

CU

Dura **25 minutos** y vale **10 puntos**. Sin apuntes, formularios ni dispositivos electrónicos: sólo estas hojas, pluma o lápiz y goma. Prohibido copiar o hablar, también al terminar. La clase se reanuda a las **19:30**. Dos ejercicios, uno por cara.

1. Del repo a la imagen (4 puntos)

<pre>ventas/ ← estás aquí ├── Dockerfile ├── README.md ├── requirements.txt ├── src/ │ ├── cargar.py │ └── formatos.py └── entrada/ └── enero.csv</pre>	ventas/ es un repo de Python. El programa es <code>src/cargar.py</code> (usa <code>src/formatos.py</code>). En tu máquina los CSV están en <code>entrada/</code>; dentro del contenedor el programa los lee de <code>/datos</code> y escribe ahí uno limpio. Queremos la imagen <code>ventas:1</code>, así: • dentro de la imagen se trabaja en <code>/ventas</code>; ahí van <code>/ventas/requirements.txt</code> y <code>/ventas/src/</code>; • al arrancar, el contenedor corre <code>python src/cargar.py</code> parado en <code>/ventas</code>; • <code>entrada/</code> no entra a la imagen: al correr, se monta en <code>/datos</code>.
--	--

Completa el Dockerfile. En cada COPY, el origen es una ruta **relativa** a `ventas/` (el contexto del build) y el destino es una ruta **absoluta** dentro de la imagen. **(1 punto, 0.25 c/u)**

FROM	python:3.12-slim
WORKDIR	(a)_____
COPY	requirements.txt (b)_____
RUN	pip install -r requirements.txt
COPY	(c)_____ /ventas/src
CMD	(d)_____

Escribe cada comando completo. Tu terminal está en `ventas/`.

e) Construye la imagen `ventas:1`. **(0.5)**

\$

f) Abre una terminal `bash` en un contenedor **nuevo** de `ventas:1`, que se borre al salir, para revisar `/ventas` (escribe sólo el comando que abre la terminal). **(0.5)**

\$

g) El programa lee los datos de `/datos`. Corre `ventas:1` en un contenedor que se borre al terminar y que monte tu carpeta `entrada/` en `/datos`. **(0.5)**

\$

h) Con el Dockerfile ya completo, ¿qué archivos del repo quedan **dentro de la imagen**? Márcalos con **X**. **(0.5)**

☐	Dockerfile	☐	README.md	☐	requirements.txt
☐	src/cargar.py	☐	src/formatos.py	☐	entrada/enero.csv

i) Alguien entra a la carpeta `ventas/src/` (`cd src`) y desde ahí corre `docker build -t ventas:1 -f ../Dockerfile .`. ¿El build termina bien? ¿Por qué? **(0.5)**

j) En g), el programa escribe `/datos/limpio.csv` y luego el contenedor se borra. ¿Dónde existe ahora ese archivo: en tu carpeta `entrada/`, en la imagen o en ningún lado? ¿Y si lo hubiera escrito en `/ventas/limpio.csv`? (mismas opciones) **(0.5)**

Examen B · 2. ¿Qué cambió? (6 puntos, 0.5 c/u)

En cfg/ hay sólo este Dockerfile y color.txt, cuyo texto es la palabra **rojo**. No existen imágenes, contenedores, volúmenes ni caché previos. Los comandos se corren **en orden**, seguidos y sin pausas (ningún sleep 600 alcanza a terminar), todos desde cfg/: cada fila parte de lo que dejó la anterior. Contesta con **una palabra o frase corta**; si crees que un comando falla, escribe **error**.

```
FROM alpine:3.20
WORKDIR /cfg
COPY color.txt .
CMD ["cat", "color.txt"]
```

#	Comandos (en orden)	Pregunta	Respuesta
1	<pre>docker build -t color:1 . docker run -d --name k1 -v "\$(pwd)":/cfg color:1 sleep 600 echo verde > color.txt docker exec k1 cat color.txt</pre>	¿Qué imprime el exec?	
2	<pre>docker run --rm color:1</pre>	¿Qué imprime?	
3	<pre>docker build -t color:1 . docker run -d --name k2 color:1 sleep 600 docker exec k2 sh -c 'echo azul > color.txt'</pre>	¿Qué contiene color.txt en tu carpeta cfg/?	
4	<pre>docker build -t color:1 .</pre>	El paso COPY, ¿se vuelve a ejecutar o sale de la caché? ¿Por qué? (pocas palabras)	
5	<pre>docker run --rm color:1 docker exec k2 cat color.txt</pre>	¿Qué imprime el run?	
		¿Y el exec?	
6	<pre>docker exec k1 sh -c 'echo gris > color.txt'</pre>	¿Qué contiene color.txt en tu carpeta cfg/?	
7	<pre>docker run --rm color:1</pre>	¿Qué imprime?	
8	<pre>docker rm -f k1 k2</pre>	El texto azul , ¿dónde existe ahora? (tu carpeta / la imagen / en ningún lado)	
9	<pre>echo negro > otro.txt docker run --rm -v "\$(pwd)/otro.txt":/cfg/color.txt color:1</pre>	¿Qué imprime?	
10	<pre>docker build -t color:1 . docker run --rm -v bodega:/cfg color:1</pre>	¿Qué imprime el run?	
11	<pre>echo blanco > color.txt docker build -t color:1 . docker run --rm -v bodega:/cfg color:1</pre>	¿Qué imprime el run?	